Illustration 16.4 The order of constructors and destructors

```
#include<iostream>
using namespace std;
class base
{
public:
base()
{ cout<<"\nConstructor of base class..."; }
~base()
{ cout<<"\nDestructor of base class.... "; }
};
class derived:public base
{
public :
derived()
{ cout << "\nConstructor of derived ..."; }
~derived()
{ cout << "\nDestructor of derived ..."; }};
class derived1 :public derived
{
public :
derived1()
{ cout << "\nConstructor of derived1 ...";}
~derived1()
{ cout << "\nDestructor of derived1 ...";}
};
int main()
{
derived1 x;
return 0;
}
```

Output:

```
Constructor of base class...
Constructor of derived ...
Constructor of derived1 ...
Destructor of derived1 ...
Destructor of derived ...
Destructor of base class....
```

Notes

- The constructors are executed in the order of inherited class i.e., from base constructor to derived. The destructors are executed in the reverse order.
- The size of derived class object=size of all base class data members + size of all data members in derived class.

16.7 Overriding / Shadowing Base class functions in derived class

In case of inheritance there are situations where the member function of the base class and derived classes have the same name. If the derived class object calls the overloaded

Chapter 16 Page 260-274.indd 269 3/24/2020 9:23:56 AM